S

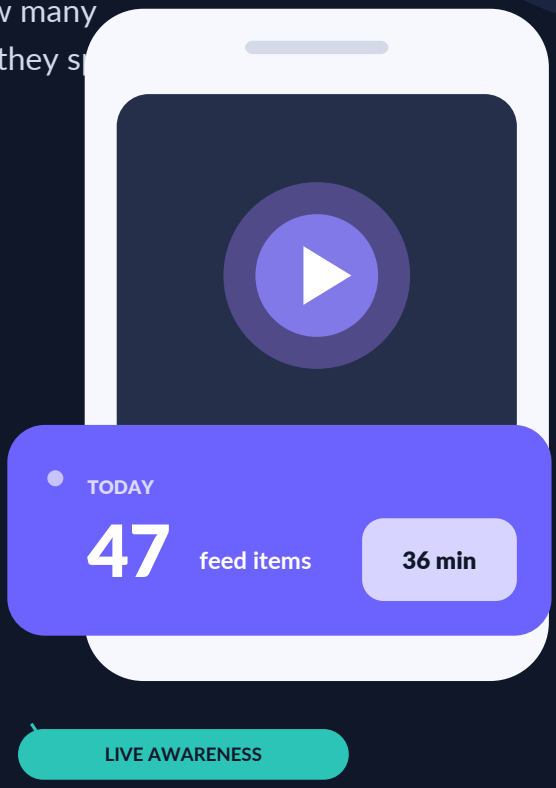
SCROLLSENSE

WORKING TITLE - NAME NOT FINAL

PROJECT CONCEPT / V1.0

MAKE SCROLLING VISIBLE.

A digital-wellbeing tracker that helps people notice how many short-form videos they encounter and how much time they s scrolling.



PYTHON-FIRST HYBRID

Android app + PC browser extension + download website

Prepared as a practical build plan | 28 October 2026

01 / PRODUCT DEFINITION

The problem is invisible scrolling

People can scroll through short-form feeds for a long time without noticing the accumulated count or time. The product makes that activity visible while it is happening.

THE PROBLEM

Autoplay and rapid feed transitions make each swipe feel small. After a long session, a person may feel mentally tired and postpone a task they meant to finish.

THE IDEA

Show a small, persistent, user-controlled counter plus time spent in supported feeds, then offer a calm nudge when the person reaches their own limit.

Product principle: awareness, not shame. The app should inform the user and help them choose to pause; it should not scold, diagnose, or secretly control their device.

MVP FEATURES

01 **Live counter**

Show today's estimated feed-item count and time in the supported app.

02 **Time tracking**

Measure app/feed session time where it can be detected reliably.

03 **Personal limits**

Optional thresholds, e.g. minutes or items, with a gentle pause reminder.

04 **Daily dashboard**

Per-platform totals, today vs. previous days, and a clear reset boundary.

05 **Local-first privacy**

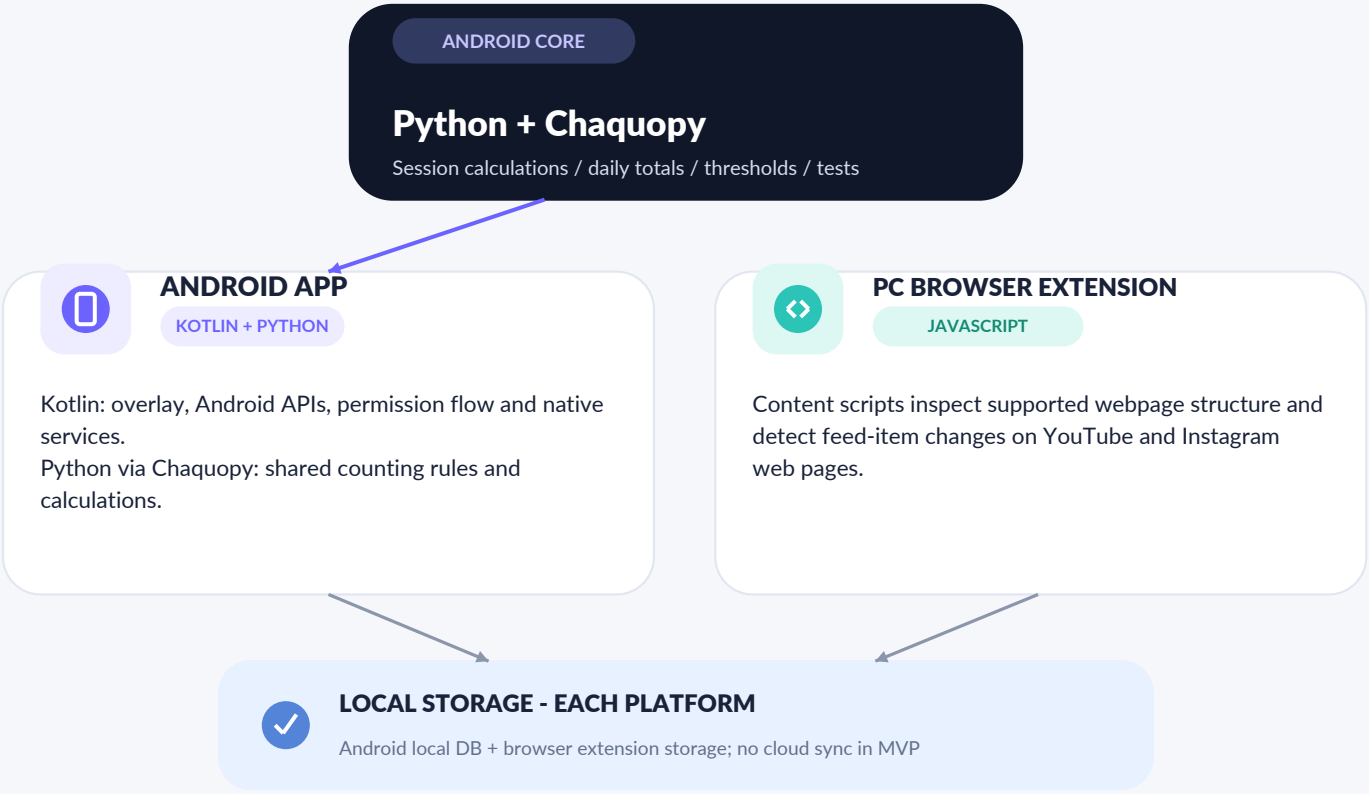
Keep the first release device-local; cloud sync remains undecided.

SCOPE NOTE

First target: Android for Instagram Reels + YouTube Shorts. PC tracking starts with a browser extension; a standalone Windows app can follow later. Counts are estimates of feed items encountered, not confirmed full watches.

What each technology does

Option B: Python runs inside the Android app through Chaquopy; Kotlin handles Android APIs, while a separate JavaScript extension tracks supported browser feeds.



DOWNLOAD WEBSITE

HTML + CSS + JavaScript
App overview, screenshots, privacy details, install instructions, and download links.

GITHUB + RELEASES

Git stores changes; GitHub hosts source code, README, issues, tags, and versioned release files.

IMPORTANT ARCHITECTURE BOUNDARY

The Android overlay and browser extension are separate integrations. The extension can inspect supported websites in a browser; it cannot count videos inside the native mobile apps. The Android app must be tested independently.

How the product should feel

The interface is designed to interrupt autopilot gently - with useful information, a tiny footprint, and user-defined limits.

01 **Set up once**

Choose supported apps, read the tracking disclosure, and grant only the permissions needed.

02 **Open a supported feed**

The app detects that a supported platform is active and starts a session timer.

03 **See the counter**

A small overlay shows today's estimated feed items plus elapsed usage time.

04 **Reach a chosen limit**

A calm prompt invites the person to pause, return to their task, or continue consciously.

05 **Review the day**

Daily totals reset at local midnight; previous summaries remain available.

FLOATING COUNTER MOCKUP

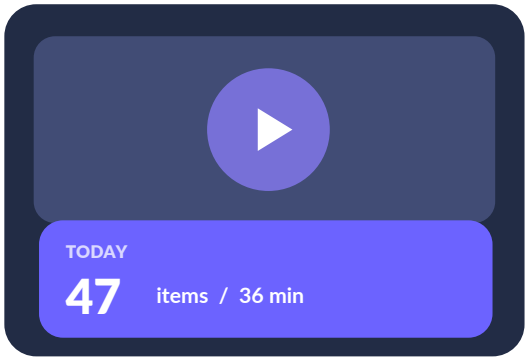


Illustration only. Actual visual placement and count accuracy must be tested on each supported app/device.

DAILY DASHBOARD MOCKUP



History view (example values)

MEASUREMENT RULE - BE PRECISE ABOUT WHAT IS COUNTED

Feed items encountered = a detected transition or item exposure, when platform structure supports it. This is not the same as a fully watched video. Time in app/feed is a separate measure and may be an estimate. The interface must label estimates honestly.

From idea to public release

Build in small, verifiable milestones. Every major stage should be committed to GitHub; do not wait until launch to start version control.

01	**Project setup** Install Python, Android Studio/SDK, Git, and an editor. Create the repository and virtual environment.	A clean, runnable project
02	**Feasibility prototype** Test detection approaches for YouTube Shorts and Instagram Reels separately. Validate permitted APIs and overlay behavior.	Evidence of what can be measured
03	**Python core** Implement session timing, daily totals, limit thresholds, data model, and unit tests.	Tested reusable logic
04	**Android shell** Build onboarding, consent/disclosure, permission explanations, local storage, and an initial dashboard.	Android development build
05	**Live counter + reminders** Integrate only feasible, policy-compliant detection and a user-controlled overlay. Add graceful fallback behavior.	First end-to-end Android MVP
06	**PC browser extension** Create content scripts for supported websites, local browser-side counters, settings, and a small popup/dashboard.	Installable extension prototype
07	**Website + documentation** Create a landing page, how-to-install guide, privacy policy, FAQ, and release notes.	Public download website
08	**Testing + hardening** Test different Android versions/devices and supported browsers; check accuracy, battery, permissions, and resets.	Release candidate
09	**Publish + maintain** Publish source and release assets on GitHub; prepare signed Android App Bundle and Play listing for review.	Public release, subject to approval

GITHUB HABIT

At each milestone: commit the working code, update README.md, record known limitations, and tag stable releases. Never commit secrets, private usage records, signing keys, or credentials.

05 / GUARDRAILS & SUCCESS

Trust is part of the product

Because the app observes activity across other apps, the design must be transparent, user-controlled, technically honest, and conservative with data.

ACCURACY FIRST

Counting is the highest-risk feature. Validate on each feed and device. If item-level detection is weak, ship an app-time-only fallback rather than fake precision.

PERMISSIONS & PLAY

Overlay permission requires explicit user approval. If an AccessibilityService is used, review declarations, in-app disclosure, affirmative consent, and Play policy before release. [1][2]

PRIVACY BY DEFAULT

Keep logs on-device for the MVP. Avoid screenshots, message content, credentials, or unrelated page data. Make tracked platforms visible and allow tracking to be paused.

DAILY RESET

Use device-local midnight as the boundary for today's totals. Reset current-day counters while keeping day-level summaries, unless the user clears them.

WHAT A SUCCESSFUL MVP LOOKS LIKE

01 Counter updates consistently on test sessions

02 Usage time matches independently checked session timing within an agreed tolerance

03 Daily reset and history remain correct across app restarts

04 Users understand what is counted and what is only estimated

PLAY STORE CHECKPOINT

As of 28 Oct 2026, Google Play says new apps and updates generally must target Android 16 / API 36 or higher from 31 Aug 2026. Confirm the current requirement again when preparing the release. [3]

FEASIBILITY GATE - A REQUIRED GO / NO-GO DECISION

Before committing to the full overlay, confirm repeatable feed-item detection, acceptable battery behavior, and a Play-policy-compliant permissions path. If item-level counting proves unreliable, ship usage-time tracking first.

REPEATABLE DETECTION | CLEAR CONSENT FLOW | USAGE-TIME FALLBACK

Terms, decisions & official sources

This brief records the current direction, not a claim that feasibility, Play review, or publication is already complete.

DECISIONS MADE SO FAR

Project	Short-form scrolling awareness tracker; working title “ScrollSense” is provisional.
First mobile platforms	Android first; target Instagram Reels and YouTube Shorts in the feasibility prototype.
Languages	Python core + Kotlin Android integration + JavaScript browser extension + HTML/CSS website.
Storage	Local-first MVP; SQLite/local persistence approach to be finalized.
Sync	Undecided; no cloud account or sync in the first prototype.
Reset	Daily totals reset at device-local midnight; historical summaries retained.

QUICK GLOSSARY

Overlay: a small window displayed on top of another app.
Content script: JavaScript that runs in a matching webpage and can read its DOM.
MVP: minimum viable product - the smallest version that tests the core promise.
AAB: Android App Bundle uploaded for Google Play distribution.

OFFICIAL TECHNICAL REFERENCES

[1] Android Developers - overlay permission
https://developer.android.com/reference/android/Manifest.permission#SYSTEM_ALERT_WINDOW

User-granted permission required to draw over other apps.

[2] Google Play - AccessibilityService API policy
https://support.google.com/googleplay/android-developer/answer/10964491

Review scope, declaration, prominent in-app disclosure, and affirmative consent if this API is considered.

[3] Google Play - target API requirements
https://support.google.com/googleplay/android-developer/answer/11926878

Current target API submission requirements; verify again at release time.

[4] Chaquopy - Python in Android
https://chaquo.com/chaquopy/doc/current/android.html

Official integration route for calling Python from a Gradle Android app.

[5] Chrome for Developers - content scripts
https://developer.chrome.com/docs/extensions/develop/concepts/content-scripts

How extension scripts interact with matching web pages.

[6] GitHub Docs - GitHub Pages
https://docs.github.com/en/pages/getting-started-with-github-pages/what-is-github-pages

Option for hosting a simple project/download website.